

From Root Objects to Certified Radical Towers

A constructive algorithm, a Wolfram Language interface,
and an executable Python reference implementation

Prepared for Vladimir Reshetnikov

8 September 2026 — Version 0.1.0

Abstract

An algebraic number is not made expressible by radicals merely by locating a small-degree polynomial factor over another algebraic field: the coefficient field must itself have a radical description, and the desired complex embedding must survive every conversion. This article turns the experiments in `FindExtension.nb` into a layered implementation. Cheap structural recognizers and optional extension hints handle favorable inputs; an explicit splitting-field algorithm supplies the general constructive route. Its essential steps are exact embedded automorphisms, a prime-index subnormal series, nonvanishing Fourier resolvents, rational coordinate reconstruction, and exact principal-branch tests. A finite procedure for the unrestricted mathematical problem is proved. The shipped code implements that procedure using SymPy, with explicit resource guards and without claiming practical performance on all solvable inputs. Successful results are radical straight-line programs, not disguised `Root` objects or trigonometric constants. The Wolfram interface independently checks each returned expression against the original input with `RootReduce`.

Implementation status. The Python backend has been executed and tested. The Wolfram package and MUnit tests are included but were not kernel-tested in this environment: both attempts to use the available Wolfram service returned HTTP 404. Exact test results and bounded-run limitations are reported in Section 9 and in the accompanying validation files.

Contents

1	The problem and the contract	3
1.1	What “any solvable Root” means here	3
2	What the notebook contributes	4
3	Structural reductions that avoid the splitting field	4
3.1	Translation and pure powers	4
3.2	Dickson polynomials and coupled radicals	5
3.3	Generalized reciprocal reductions	5
3.4	Cyclotomic expressions without hidden trigonometry	6

4	The exact obstruction: normal closure, not root-field automorphisms	6
4.1	An exact inexpensive negative test in prime degree	7
5	An explicit general construction	7
5.1	Building a normal field and its embedded automorphisms	7
5.2	Constructing a prime-index series without subgroup enumeration	8
5.3	The nonzero Fourier resolvent	9
5.4	Reconstructing the radicand and the final answer	9
5.5	Finite algorithm and correctness	10
6	Principal branches and exact certification	10
6.1	Why a radicand equation alone is insufficient	10
6.2	Zero detection before interval refinement	11
6.3	Two positive verification paths	11
7	Implementation architecture and data interchange	11
7.1	A radical straight-line language	12
7.2	Root ordering across systems	12
7.3	Dependency and operational limits	13
8	Using the package	13
8.1	Wolfram Language	13
8.2	Python	14
9	Validation and observed limitations	14
10	Complexity and the next engineering steps	15
A	General-route pseudocode	16
B	Reproducibility and license	17

1 The problem and the contract

The motivating question asks why `ToRadicals` sometimes leaves an exact `Root` unchanged even though a radical expression exists [1]. Wolfram documents precisely this possibility, while guaranteeing conversion for polynomial roots of degree at most four [2]. The useful goal is therefore not to wrap `ToRadicals` in more simplification rules, but to supply the missing construction.

The input to the core algorithm is a specified algebraic number $\alpha \in \overline{\mathbb{Q}} \subset \mathbb{C}$, with its complex embedding. Let $f \in \mathbb{Q}[x]$ be its monic minimal polynomial. A successful result is a finite expression generated from rational numbers by addition, multiplication, inversion, and rational powers with their ordinary principal complex meanings. The implementation also admits i , which is itself $(-1)^{1/2}$. Parameter-dependent algebraic functions, approximate numbers, and arbitrary transcendental constants are outside this contract.

This formulation matters in three ways. First, the input polynomial need not be irreducible: only the factor belonging to the selected root is relevant. Second, specifying a polynomial without its root embedding is insufficient to select one radical value. Third, finding an expression for some conjugate is not success for the original `Root`.

The public result distinguishes the following outcomes.

Python status	Meaning
success	A radical expression has passed an exact algebraic verification.
not_solvable	An exact obstruction proves that the minimal polynomial has a nonsolvable Galois group.
resource_limit	A configured degree or worker-time limit was reached. No conclusion about solvability follows.
unknown	A bounded recognizer failed, a backend computation failed, or verification was inconclusive.
invalid_input	The JSON request was malformed or violated the input contract. The direct Python API raises validation exceptions instead.

The Wolfram association uses corresponding capitalized status names. Its `RootToRadicals` convenience function returns an expression only on success; otherwise it returns a `Failure` carrying the structured result.

1.1 What “any solvable `Root`” means here

There is a finite, general mathematical algorithm. Its proof is given below, and the explicit-field backend is an implementation of that algorithm rather than a placeholder requiring an unimplemented Galois oracle. It does not rely on a finite table of transitive groups. However, it constructs a splitting field as an absolute number field and can be extremely expensive. A default field-degree limit of 64 and a CLI worker limit of 180 seconds deliberately make the default invocation a resource-bounded procedure.

Removing those limits removes these policy cutoffs; it does not provide a practical time bound, eliminate expression swell, or establish that every underlying CAS routine is bug-free. The completeness theorem is about the algorithm under terminating exact algebraic primitives. The validation claims are about the concrete inputs actually executed.

2 What the notebook contributes

The supplied archive contains `FindExtension.nb`. Its input cells include several complementary searches rather than a single closed algorithm. `findExtension` imposes divisibility by a generic quadratic $x^2 + px + q$, eliminates variables, and tries to select a useful coefficient. `findExtension3` repeats the idea with a cubic factor. `project` tests whether real or imaginary parts of conjugates have lower minimal-polynomial degree. The `factor` and `solve` helpers then select factors or roots using `PossibleZeroQ`. Finally, `fuzz` searches sums of randomly selected conjugates, reconstructing candidate algebraic numbers from high-precision approximations.

These are good ways to discover structure. A coefficient of a factor over an extension field is an invariant of a subset of roots, hence a natural resolvent value. Lower-degree real or imaginary projections can expose subfields. Numerically guessed sums can reveal useful fixed-field generators. The notebook also contains manually chosen radical and cyclotomic extensions, and several much larger inverse-beta-related examples.

The missing invariant is a *certified, decreasing field-theoretic task*. An algebraic coefficient of degree three may be easy to radicalize, but a coefficient of degree five may not be. Repeatedly choosing a smaller-looking coefficient need not produce a terminating chain. A low-degree factorization also does not specify the correct principal branches of its formulas.

The implementation retains the useful part as *optional extension hints*. Hints must already be radical expressions. Factoring over the field they generate is exact, and every proposed answer is verified independently. The general route replaces random subset searches with traces and character projectors on an explicitly computed normal extension. At a prime-index step, a nonzero projector is guaranteed to occur among a finite power-basis list.

Some notebook details should not be promoted into a package contract: selection by `[[1]]` assumes a nonempty candidate list; `fuzz` has an unbounded search loop; `PossibleZeroQ` is not used here as an equality certificate; and one complexity-function name appears as `Simplify`‘`SimplifyCount`. The delivered code avoids undocumented simplifier complexity functions. It does not evaluate the original notebook or claim that all its large examples have been solved. A source manifest and selected regression polynomials are included in `notes/`.

3 Structural reductions that avoid the splitting field

A general algorithm should not make a sextic pay for a degree-24 splitting field when a cubic substitution will do. Recognition is performed over $\mathbb{Q}$, with exact coefficient comparisons. Candidate ordering may use approximations, but acceptance does not.

3.1 Translation and pure powers

For a monic polynomial of degree n , translation $x = y - c_{n-1}/n$ removes the next-to-leading coefficient. If the result is $y^n - b$, enumerate

$$y_k = \zeta_n^k b^{1/n}, \quad 0 \leq k < n, \quad \zeta_n = (-1)^{2/n}.$$

The branch of $b^{1/n}$ is principal, and the root-of-unity factors enumerate all alternatives. In particular, a negative real radicand of odd order is not silently interpreted as a real root by the complex-power operation.

If all nonconstant exponents have a common divisor $g > 1$, solve the reduced polynomial in y^g , and again enumerate every phase of each g -th root. The Python recognizer also handles rational polynomial compositions when all inner components can be inverted in degree at most four.

3.2 Dickson polynomials and coupled radicals

Define

$$D_0(x, a) = 2, \quad D_1(x, a) = x, \quad D_n(x, a) = xD_{n-1}(x, a) - aD_{n-2}(x, a).$$

Induction gives the central identity

$$D_n(u + a/u, a) = u^n + (a/u)^n. \quad (1)$$

For $D_n(x, a) = b$, solve $U^2 - bU + a^n = 0$, take one nonzero solution U , and put

$$u = U^{1/n}, \quad x_k = \zeta_n^k u + \frac{a}{\zeta_n^k u}.$$

When $a \neq 0$, the chosen U is nonzero. The two summands are coupled by $uv = a$. Choosing two unrelated principal n -th roots instead can break that condition and produce an incorrect answer.

For the quintic in the question,

$$5x^5 - 25x^3 + 25x + 6 = 5 \left(D_5(x, 1) + \frac{6}{5} \right).$$

Thus, with all powers principal,

$$u = \left(\frac{-3 + 4i}{5} \right)^{1/5}, \quad \boxed{\alpha = u + u^{-1}}. \quad (2)$$

Since u has unit modulus and argument between zero and $\pi/5$, this is the largest real root, namely the question's Wolfram index 5. Its other roots are obtained by the same phase enumeration. The radical expression was verified against the specified algebraic number, not only substituted into the quintic.

3.3 Generalized reciprocal reductions

Suppose a monic polynomial of degree $2d$ has coefficients satisfying

$$c_{d-j} = a^j c_{d+j} \quad (1 \leq j \leq d), \quad a \in \mathbb{Q}^\times.$$

In particular, $c_0 = a^d$, so there are at most two rational candidates for a . Then

$$f(x) = x^d h(x + a/x), \quad h(t) = c_d + \sum_{j=1}^d c_{d+j} D_j(t, a). \quad (3)$$

After solving $h(t) = 0$, recover $x = (t \pm \sqrt{t^2 - 4a})/2$. Both signs must be retained until the selected embedding is certified. The familiar reciprocal substitution $x + 1/x$ is just the case $a = 1$; allowing $a = -1$ is essential for the sextic.

Indeed,

$$\begin{aligned} f(x) &= x^6 + x^4 - x^3 - x^2 - 1 \\ &= x^3 \left((x - 1/x)^3 + 4(x - 1/x) - 1 \right). \end{aligned} \quad (4)$$

Let

$$v = \left(\frac{9 + \sqrt{849}}{18} \right)^{1/3}, \quad t = v - \frac{4}{3v}. \quad (5)$$

Then $t^3 + 4t - 1 = 0$. The cubic is strictly increasing on the real line, so it has exactly one real root. The sextic therefore has exactly two real roots, one of each sign, and the positive one is

$$\alpha = \frac{t + \sqrt{t^2 + 4}}{2}. \quad (6)$$

This is the original Wolfram `Root[... , 2]`. It needs only a cubic radical and square roots, with no auxiliary search for a general quartic field.

3.4 Cyclotomic expressions without hidden trigonometry

For a cyclotomic polynomial Φ_m , emit $\zeta_m^k = (-1)^{2k/m}$ for $\gcd(k, m) = 1$. This is an actual radical expression under the contract; neither $\cos(2\pi/m)$ nor a new `Root` is retained in the output.

The Python recognizer also detects maximal real cyclotomic fields. For $m \geq 3$, eliminating z between $\Phi_m(z) = 0$ and $z^2 - tz + 1 = 0$ gives the square of the minimal polynomial of $\zeta_m + \zeta_m^{-1}$. The square appears because k and $-k$ give the same trace. An exact squarefree-part comparison recognizes the polynomial and emits these traces using powers of -1 .

The searches are finite. From

$$\frac{\varphi(m)^2}{m} = \prod_{p^e \parallel m} p^{e-2}(p-1)^2 \geq \frac{1}{2}$$

we have $m \leq 2n^2$ when $\varphi(m) = n$, and $m \leq 8n^2$ when $\varphi(m) = 2n$. Every factor on the left is at least one except possibly the factor for 2^1 , which is $1/2$. This proves the bounds used by the recognizers. These searches recognize cyclotomic generators and their maximal real traces, not arbitrary generators of every abelian extension.

4 The exact obstruction: normal closure, not root-field automorphisms

Let L be the splitting field of the minimal polynomial f , in the chosen algebraic closure, and $G = \text{Gal}(L/\mathbb{Q})$. The classical criterion is

$$\alpha \text{ is expressible by radicals over } \mathbb{Q} \iff G \text{ is solvable.} \quad (7)$$

For background on radical extensions, normal closures, and the Galois correspondence, see Milne [3]. This criterion concerns the minimal polynomial of the *particular input*, not every irreducible factor of a possibly larger input polynomial.

A frequent wrong shortcut is to compute $\text{Aut}(\mathbb{Q}(\alpha)/\mathbb{Q})$ instead of the Galois group of its normal closure. For an irreducible polynomial that automorphism group can be trivial even when the normal closure has a large nonsolvable group. A second wrong shortcut is to infer radical solvability merely from the degree being a product of small primes. Degree alone supplies no such criterion.

For completeness, the necessity in (7) can be understood by normalizing a radical tower: adjoin the necessary roots of unity and all conjugates of the successive radicals. The resulting normal extensions have abelian successive kernels. Their composite has a solvable Galois group, and so does the quotient acting on the splitting field of f . Conversely, the construction in the next sections explicitly realizes a solvable normal extension by a radical tower.

4.1 An exact inexpensive negative test in prime degree

Proposition 4.1. *Let $f \in \mathbb{Q}[x]$ be irreducible of prime degree n . At a prime of good squarefree reduction, the degrees of its irreducible modular factors are the cycle lengths of an element of its Galois group. If those lengths are not of one of the forms*

$$1^n, \quad n, \quad 1 \, d^{(n-1)/d} \quad (d > 1, \, d \mid n-1),$$

then f is not solvable by radicals.

Proof. A solvable transitive group of prime degree has a minimal nontrivial normal subgroup N that is elementary abelian. Normal orbits are blocks, so N is transitive. An abelian transitive permutation group is regular: a point stabilizer fixes every point, hence is trivial in this faithful action. Thus $|N| = n$ and $N \simeq C_n$. The full group lies in the normalizer of this regular cyclic subgroup, namely the affine group $x \mapsto ax + b$ of $\mathbb{F}_n$. Translations have an n -cycle or are the identity. For $a \neq 1$, there is one fixed point and every other orbit has length the multiplicative order of a . Combining this classification with the good-prime Frobenius factorization theorem proves the claim. $\square$

The implementation tries twelve small rational primes. A failure to find an obstruction means nothing negative. For example,

$$x^5 - x - 1 \equiv (x^2 + x + 1)(x^3 + x^2 + 1) \pmod{2}$$

is squarefree and has cycle type $(2, 3)$, which excludes all solvable transitive quintic groups. For $x^7 - x - 1$, reduction modulo 3 gives factor degrees $(2, 5)$, likewise an obstruction. Irreducibility is not guessed: the caller has already replaced the input by its minimal polynomial.

The optional SymPy `galois_group` call is a second negative shortcut for degrees at most six, its documented range [5]. Its returned abstract permutations are never applied to a numerically sorted list of roots. The constructive backend computes an embedded action itself.

5 An explicit general construction

5.1 Building a normal field and its embedded automorphisms

The reference backend enumerates all roots of f exactly and successively adjoins roots not already in the current field. This gives the splitting field L . Containment and annihilation of all distinct roots

are checked. The use of a primitive element lets every field element be represented by a rational coefficient vector modulo one irreducible polynomial.

Write $g = [L : \mathbb{Q}] = |G|$, and set

$$m = \text{rad}(g) = \prod_{p|g} p, \quad B = \mathbb{Q}(\zeta_m), \quad M = LB. \quad (8)$$

Here M is again normal over $\mathbb{Q}$. Represent it as $\mathbb{Q}(\theta)$, with $D = [M : \mathbb{Q}]$ and monic minimal polynomial P of θ . Factoring P over M must give exactly D distinct linear factors. Their roots are precisely the possible images of θ under all $\mathbb{Q}$ -automorphisms of M .

For each such image θ' , the automorphism acts by reduced Horner substitution:

$$\sigma_{\theta'} \left(\sum_{j=0}^{D-1} q_j \theta^j \right) = \sum_{j=0}^{D-1} q_j (\theta')^j.$$

Keep the automorphisms fixing ζ_m . They form $H = \text{Gal}(M/B)$. Their composition table is computed by applying one substitution to the other's image of θ ; closure, identity, inverses, and the degree equality $|H|\varphi(m) = D$ are checked.

Lemma 5.1. *H is solvable if and only if G is solvable. Every prime dividing $|H|$ divides m .*

Proof. Restriction identifies H with $\text{Gal}(L/L \cap B)$, a subgroup of G . Since $B/\mathbb{Q}$ is abelian, $(L \cap B)/\mathbb{Q}$ is abelian and Galois. Thus this subgroup is normal, with abelian quotient. A group is solvable precisely when such a normal subgroup is solvable. Its order divides $|G|$, proving the assertion about primes. $\square$

This avoids an unnecessary recursive expansion of the cyclotomic base. The output language already expresses ζ_m by the single rational power $(-1)^{2/m}$. Primes dividing $\varphi(m)$ do not need to be added merely to provide a separate prime-radical decomposition of that constant. The algorithm is not minimizing the degree of M .

5.2 Constructing a prime-index series without subgroup enumeration

For a finite solvable group H , compute its commutator subgroup H' . Every subgroup containing H' is normal, because H/H' is abelian. Starting at H' , add an element whenever the generated subgroup remains proper in H . When no further addition is possible, the resulting N is maximal among proper subgroups containing H' . Consequently H/N is cyclic of prime order.

Repeat inside N , obtaining

$$H = H_0 \triangleright H_1 \triangleright \cdots \triangleright H_s = 1, \quad [H_{j-1} : H_j] = p_j \text{ prime.} \quad (9)$$

Each subgroup is normal in its immediate predecessor; normality in the entire original group is not required. If a nontrivial subgroup equals its own commutator subgroup, the derived series has stabilized nontrivially and the algorithm proves nonsolvability instead.

5.3 The nonzero Fourier resolvent

Fix one step $H_0 \triangleright N$, of index p , and put $K = M^{H_0}$, $E = M^N$. Then E/K is cyclic of degree p , and $\zeta_p = \zeta_m^{m/p} \in B \subset K$. Choose $\sigma \in H_0 \setminus N$. Its restriction generates $\text{Gal}(E/K)$.

For $0 \leq k < D$, form

$$T_k = \sum_{\tau \in N} \tau(\theta^k), \quad R_k = \sum_{j=0}^{p-1} \zeta_p^{-j} \sigma^j(T_k). \quad (10)$$

This is a finite, deterministic replacement for random sums of roots.

Lemma 5.2. *At least one R_k is nonzero. Every nonzero $R = R_k$ satisfies*

$$\sigma(R) = \zeta_p R, \quad R^p \in K, \quad E = K(R).$$

Proof. The trace from M to E is a surjective $\mathbb{Q}$ -linear map: on E it is multiplication by $|N|$. Hence the T_k span E over $\mathbb{Q}$. The character projector $\sum_j \zeta_p^{-j} \sigma^j$ is not the zero map on E : distinct field automorphisms are linearly independent over the field. Thus it cannot vanish on all the spanning vectors T_k .

For any $T_k \in E$, $\sigma^p(T_k) = T_k$, so shifting the sum gives $\sigma(R) = \zeta_p R$. Therefore R^p is fixed by σ and by N , hence lies in K . Since $R \neq 0$ and $\zeta_p \neq 1$, $R \notin K$. The prime extension degree forces $K(R) = E$. $\square$

This is the classical cyclic-to-radical construction in an explicit embedded field. Related, substantially more efficient Galois-based constructions are developed by Fieker and Sutherland [4]. The present backend does not implement their local reconstruction machinery; it deliberately uses exact absolute-field arithmetic to make the construction executable with its single dependency.

5.4 Reconstructing the radicand and the final answer

Maintain a $\mathbb{Q}$ -basis $b_1, \dots, b_e$ of the current fixed field and a parallel list of rational expressions in already emitted radical registers. Initially this is

$$1, \zeta_m, \dots, \zeta_m^{\varphi(m)-1}.$$

After finding R , express $a = R^p$ in that basis by exact rational linear algebra. Coefficient vectors are stored in the absolute power basis of θ ; solving the resulting rectangular linear system gives the unique rational coefficients of a .

Choose the principal root $C = a^{1/p}$ among the values $R\zeta_p^{-k}$, as detailed in Section 6. Since $\zeta_p \in K$, replacing R by C does not change the field it generates. Emit a register assignment $r_j = a^{1/p}$, and replace the basis by

$$\{b_i C^k : 1 \leq i \leq e, 0 \leq k < p\}.$$

This is a basis of the next field: $[E : K] = p$. At the end of the series, the basis spans all of M , so the target α has unique rational coordinates in it. Substituting the parallel register expressions gives the radical output.

5.5 Finite algorithm and correctness

Theorem 5.3. *Assuming terminating exact algorithms for algebraic-number representations, number-field arithmetic and factorization, rational linear algebra, and algebraic root isolation, the unrestricted explicit-field construction decides whether a specified $\alpha \in \overline{\mathbb{Q}}$ is expressible by radicals. On a positive input it constructs a principal-branch radical straight-line program equal to that exact embedding of α .*

Proof. There are finitely many roots to adjoin, so the normal field is obtained after finitely many number-field operations. The primitive polynomial has finite degree D , so its factorization enumerates all automorphisms. Finite-group closure and commutator computations terminate. A nontrivial stable derived subgroup is an exact negative certificate, by the equivalence with G .

Otherwise the strictly decreasing series (9) is finite. At each step, the projector lemma guarantees success among at most D traces. Rational coordinate reconstruction succeeds because the radicand lies in the current fixed field. The branch tests below terminate, including on the boundary rays. The basis invariant then proves that the last coordinate reconstruction yields exactly α . Conversely, the nonsolvability criterion excludes a radical expression in the negative case. $\square$

The proof does not assert a useful polynomial-time bound. Nor does it assert that the resulting expression is shortest, has least radical depth, or lies in a minimal-degree radical extension. These are separate optimization tasks.

6 Principal branches and exact certification

6.1 Why a radicand equation alone is insufficient

An algebraic identity $R^p = a$ provides p possible values, not a proof that R is the value of the expression $a^{1/p}$. The ambiguity is particularly visible for $p = 3$ and $a = -1$: the real root -1 is not the principal complex cube root.

For nonzero a , the principal p -th root is characterized by

$$C^p = a, \quad -\frac{\pi}{p} < \operatorname{Arg} C \leq \frac{\pi}{p}. \quad (11)$$

The asymmetry of the two boundary inequalities is intentional.

For $p = 2$, the exact sector predicate is

$$\Re C > 0 \quad \text{or} \quad (\Re C = 0 \text{ and } \Im C > 0).$$

For an odd prime, let $\eta = -\zeta_p^{(p+1)/2} = e^{\pi i/p}$. Then (11) is equivalent to

$$\Im(C\eta) > 0 \quad \text{and} \quad \Im(C/\eta) \leq 0. \quad (12)$$

To verify this, intersect the two open/closed half-planes after rotation. Their intersection is exactly the sector $(-\pi/p, \pi/p]$; there is no second component at the negative real axis. No transcendental argument function is evaluated by the implementation.

6.2 Zero detection before interval refinement

All field elements lie in the normal field M , so complex conjugation is an exact automorphism. For $z \in M$,

$$\Re z = 0 \iff z + \bar{z} = 0, \quad \Im z = 0 \iff z - \bar{z} = 0.$$

These tests are performed by rational coefficient-vector equality. Only a component proved nonzero is passed to a sign-refinement loop.

The primitive embedding θ is isolated as an exact algebraic root. Rational rectangles enclosing it are refined, and a polynomial expression in θ is evaluated by interval Horner arithmetic. Interval multiplication uses all endpoint products; complex multiplication uses the usual real and imaginary formulas with outward rational intervals. No machine rounding is involved. Because the component is known to be nonzero and the rectangles shrink to θ , the enclosing component interval eventually lies strictly on one side of zero.

This order of operations is crucial. A pure interval procedure may refine forever on the positive boundary ray of (11), where one component is exactly zero. The tests include both boundary rays for cube roots and both imaginary-axis choices for square roots.

6.3 Two positive verification paths

Structural candidates. For a proposed expression e , the implementation first verifies the radical grammar and computes its exact minimal polynomial. This must equal the monic minimal polynomial of the target. Only then is `Poly.same_root(e, target)` used to identify the embedding. The documented operation compares two roots of a polynomial [6]; its precondition must not be replaced by numerical proximity to a root.

For example, $\sqrt{2} + 10^{-100}$ is extremely close to $\sqrt{2}$, but is not another representation of it. It is rejected before the root-separation comparison. A wrong conjugate, such as $-\sqrt{2}$, passes the minimal- polynomial test but fails the selected-root test.

General towers. The in-memory certificate replays the output independently of the Galois construction. It checks the root-of-unity register, every equation $C_j^{p_j} = a_j$, every principal-sector condition, and the final equality to α in the exact field. Conjugation, the cyclotomic register, and the final target embedding are independently anchored by exact annihilation followed by a known-root separation comparison; a field-membership conversion is not trusted merely to have chosen some conjugate. Thus a bug in group selection or a bad projector cannot be hidden by a merely plausible numerical answer. The certificate reuses the number-field and root-isolation infrastructure; it is an exact CAS certificate, not a Lean-checked proof.

The JSON certificate section is explicitly a *summary*, not a standalone serialized proof. It records the absolute field modulus, primitive expression, prime indices, subgroup orders, and generator vectors for inspection. A fresh process should either rerun the construction, verify the expanded radical expression, or use the Wolfram exact check. No text in that summary is executed.

7 Implementation architecture and data interchange

The Python code is divided along mathematical trust boundaries:

Module	Responsibility
<code>program.py</code>	Radical grammar, straight-line programs, whitelisted JSON DAG, Wolfram rendering.
<code>groups.py</code>	Exact multiplication-table closure, commutators, derived series, prime-index series.
<code>field.py</code>	Embedded number-field arithmetic, automorphisms, coordinates, rational interval signs.
<code>solver.py</code>	Structural recognition, hint fields, negative tests, general tower construction, replay certificates.
<code>cli.py</code>	Strict request parsing, selected-root input, subprocess isolation and cancellation.

7.1 A radical straight-line language

A program consists of ordered assignments $r_j := E_j$, with each E_j using only earlier registers, followed by an output expression. This preserves sharing: a large radicand need not be copied at every use. The JSON transport has only the operations `Q`, `I`, `Add`, `Mul`, and `Pow`; rational numerators and denominators are decimal strings, and every node reference points strictly backward. Rational-power denominators are positive. Undefined and nonfinite values are rejected.

The Wolfram rendering uses nested `With`, not one simultaneous `With` binding list whose right sides incorrectly assume sequential scope. The JSON decoder builds arithmetic expressions directly. It never calls `ToExpression` on backend output and never executes an arbitrary returned source string. The human-readable `wolfram_program` is informational.

Expanding the DAG can still cause expression swell. The Python `program.expression` size check is a convenience guard after substitution, not a hard memory sandbox. The worker deadline is the reliable cancellation boundary for expensive algebraic primitives; memory usage is not hard-limited by this release.

7.2 Root ordering across systems

Python's direct API accepts an exact SymPy expression; the JSON API accepts a polynomial with descending rational coefficients and a zero-based SymPy root index. Wolfram uses its own root indexing, which must not be converted merely by subtracting one for arbitrary complex inputs.

The Wolfram interface instead transmits a rationalized high-precision complex approximation as a search hint. Python selects a likely conjugate, constructs and certifies a result for that conjugate, and returns it. Wolfram then checks

```
RootReduce[returnedExpression - originalRoot] == 0
```

with a separate verification limit. If this does not succeed, it increases the hint precision and retries. An incorrect hint can therefore cause extra work or an `Unknown` result, but not an accepted expression for the wrong root. A negative Galois conclusion is independent of this conjugate choice because the transmitted polynomial is the input's irreducible minimal polynomial.

7.3 Dependency and operational limits

The executed backend used Python and SymPy 1.14.0; the requirements pin that SymPy version for reproducibility. It needs neither SageMath, GAP, Magma, PARI/GP, nor an undocumented Wolfram function. SymPy supplies exact algebraic primitives, while the arbitrary-degree embedded group construction is in this package. Its optional tabulated-degree Galois precheck is not its general route.

The direct Python call has no implicit wall-clock deadline. The CLI starts a worker subprocess and kills and reaps it on timeout, including on Windows. The timeout covers the worker's algebraic work, not the parent interpreter's initial startup. A field-degree check occurs *after* a primitive-field construction returns; an expensive field compositum may therefore consume the time budget before its degree can be checked. These distinctions are visible in the returned statuses.

8 Using the package

8.1 Wolfram Language

After extracting the archive, load the package by its path:

```
Get["/path/to/RootRadicals/wolfram/RootRadicals.wl"];

a = Root[-1 - #^2 - #^3 + #^4 + #^6 &, 2];
e = RootToRadicals[a];
RootReduce[e - a] === 0
```

The two original examples have native structural paths. The general backend requires Python with the accompanying dependency installed. A Windows interpreter may be supplied explicitly; no shell quoting is required because `RunProcess` receives an argument list.

```
r = RadicalizeRoot[a,
  "PythonExecutable" -> "C:\\Python311\\python.exe",
  "UseNative" -> False,
  "TimeLimit" -> 180,
  "MaxFieldDegree" -> 64];

r["Status"]
r["Expression"] (* only when Status is Success *)
```

Use `Method -> "Galois"` to force the explicit-field construction, or `Method -> "Structural"` to stop after recognizers and hints. The strings `"auto"`, `"galois"`, and `"structural"` are also accepted. To remove the two backend policy limits:

```
RadicalizeRoot[a, Method -> "Galois",
  "TimeLimit" -> Infinity, "MaxFieldDegree" -> Infinity]
```

Wolfram's separate native and verification limits remain in force unless changed. The default verification limit is 60 seconds, and the default native limit is 15 seconds. A returned `Success` always means the exact final Wolfram equality check succeeded.

An extension hint is an expression, not a polynomial that defines an unknown coefficient field:

```
RadicalizeRoot[a, "ExtensionHints" -> {Sqrt[2], Sqrt[3]}]
```

Hints are optional accelerators and cannot make a nonsolvable number solvable.

8.2 Python

From the extracted project root:

```
python -m pip install -e ./python
python -m unittest discover -s tests -v
```

Then:

```
import sympy as s
from rootradicals import radicalize
x = s.Symbol("x")
a = s.CRootOf(x**6 + x**4 - x**3 - x**2 - 1, 1)
r = radicalize(a)
assert r.status == "success" and r.verify()
print(r.expression)
print(r.program.wolfram())
```

For the general route:

```
r = radicalize(s.CRootOf(x**8 + 1, 7), method="galois")
assert r.verify()
print(r.details)           # field degrees and prime indices
print(r.program.assignments) # shared radical registers
print(r.program.output)
```

The CLI avoids arbitrary expression parsing. A minimal request is:

```
{"coefficients": [["1", "1"], ["0", "1"], ["-2", "1"]],
 "root_index": 1, "method": "auto", "time_limit": 180,
 "max_field_degree": 64}
```

Pass that JSON on standard input to `python python/driver.py`. Setting `time_limit` and `max_field_degree` to JSON null removes the corresponding guards. The direct Python API is for trusted SymPy expressions; it is not an untrusted-text parser. Use the strict JSON protocol for external inputs.

9 Validation and observed limitations

The final Python regression run executed **46 tests**, with **0 failures and 0 errors**, using Python 3.13.5 and SymPy 1.14.0. The separate development extended-smoke run completed 27 conversions; all passed exact verification and independent minimal-polynomial/selected-root checks. The Wolfram MUnit tests were not executed: the available Wolfram service returned HTTP 404.

The tests cover structural conversion, the two original roots, translated binomials, power substitutions, cyclotomic and real-cyclotomic expressions, reducible and repeated-root inputs, extension-hint membership, negative Frobenius witnesses, resource-limit semantics, finite-group series, exact branch boundaries, corrupted certificates, and strict JSON decoding. Several general-route outputs are also verified by recomputing the minimal polynomial of the expanded radical expression, rather than relying only on tower replay.

Input / route	Normal-field sizes	Relevant observation
Original sextic / structural	Not constructed	Reciprocal reduction to a cubic.
Original quintic / structural	Not constructed	Coupled fifth roots, all phases available.
$x^3 - 2$ / forced Galois	$[L : \mathbb{Q}] = [M : \mathbb{Q}] = 6$	One cubic step over the cyclotomic base.
$x^4 + 1$ / forced Galois	$[L : \mathbb{Q}] = [M : \mathbb{Q}] = 4$	Two quadratic steps; branch replay.
$x^8 + 1$ / forced Galois	$[L : \mathbb{Q}] = [M : \mathbb{Q}] = 8$	Three quadratic steps; 14-node output DAG in the recorded CLI run.
$x^5 - x - 1$ / automatic	Not constructed	Exact modular obstruction (2, 3).
$x^7 - x - 1$ / automatic	Not constructed	Exact modular obstruction (2, 5).

An important performance limitation was also recorded. Forcing the general route on

$$x^5 + x^4 - 4x^3 - 3x^2 + 3x + 1$$

hit a 90-second worker limit. This polynomial is solvable: it is the minimal polynomial of $2 \cos(2\pi/11)$. The real-cyclotomic structural path handles it without that construction. The forced timeout is included rather than silently discarded; it demonstrates why algorithmic completeness and practical coverage must be reported separately.

Complex conjugates of the original sextic are substantially more expensive under expanded minimal-polynomial verification than its two real roots. The separate extended-smoke record reports those actual runs. The package does not claim comprehensive testing of every polynomial in the notebook, especially the large coefficient and high-degree inverse-beta examples. The selected notebook fixtures are retained as future regression targets, not falsely marked as passing tests.

The Wolfram source received static review and a delimiter check. That is not kernel execution. The supplied MUnit tests should be run in a local Wolfram installation before treating the frontend as validated in that environment. The Python implementation is the executed reference artifact.

10 Complexity and the next engineering steps

The bottleneck is the representation of the normal field, not the existence of radicals. If $n = \deg f$, then $|G| \leq n!$, and

$$D = [M : \mathbb{Q}] \leq |G| \varphi(\text{rad } |G|).$$

This is an upper bound, not a forecast of typical field sizes. It nevertheless explains why a root of modest degree can require an unwieldy absolute field. The primitive element can have large coefficients even when D is small. Factoring a degree- D polynomial over a degree- D field,

constructing a full multiplication table, and repeated rational linear systems magnify this cost. The simple dense coordinate solver is also repeated rather than incrementally maintained.

There are clear improvements that preserve the proof architecture. Field bases could be reduced and coordinate factorizations cached. Several projector values could be tried and scored for coefficient height instead of choosing the first nonzero value. Small intermediate fixed fields could be constructed using certified resolvents before forming a full absolute splitting field. Repeated requests for different conjugates should share the normal-field and tower construction, with only the final coordinate vector changing. Native subfield and cyclotomic-conductor algorithms could replace bounded pattern recognition for abelian examples.

For a production implementation aimed at difficult large-degree inputs, Galois computation with local representations, invariant bounds, and exact reconstruction is the more promising direction. Fieker and Sutherland's work is a primary reference for that engineering path [4]. Magma's documented `SolveByRadicals` and `CyclicToRadical` interfaces provide an existing implementation in that ecosystem [7]. This package does not depend on Magma and does not claim to match its performance.

Finally, minimizing radical depth or expression length requires a different objective from merely constructing a radical tower. Different subgroup chains, generator choices, and cyclotomic bases can produce very different expressions. The supplied algorithm proves and computes existence, protects the selected embedding, and exposes a straight-line representation suitable for later denesting and simplification; it does not claim a canonical or optimal normal form.

A General-route pseudocode

```

Radicalize(alpha):
  f := exact minimal polynomial of alpha over Q
  try structural candidates and certified hint-field reductions
  try exact inexpensive nonsolvability obstructions
  L := embedded splitting field of f
  m := product of distinct primes dividing [L:Q]
  M := L(zeta_m); B := Q(zeta_m)
  enumerate exact automorphisms of M fixing zeta_m
  H := their exact multiplication-table group
  if H is not solvable: return NotSolvable
  chain := H = H_0 > ... > H_s = 1, prime normal indices
  basis := [1, zeta_m, ..., zeta_m^(phi(m)-1)]
  registers := [r_0 = (-1)^(2/m)]
  for (H_j, N = H_(j+1)) in chain:
    p := |H_j| / |N|; choose sigma in H_j minus N
    for k = 0, ..., [M:Q]-1:
      T := sum(tau(theta^k), tau in N)
      R := sum(zeta_p^(-i) * sigma^i(T), i=0, ..., p-1)
      if R != 0: break
    a := reconstruct R^p in the current basis
    C := the unique R*zeta_p^(-i) in the principal sector
    emit r_(j+1) := a^(1/p), using earlier registers
    basis := [b*C^i for i=0, ..., p-1 for b in basis]
  output := reconstruct alpha in the final basis
  replay radical equations, principal sectors, and target equality
  return Success(registers, output)

```


B Reproducibility and license

The archive supplies all newly written code, the article source, the compiled article, examples, tests, machine-readable validation, and an MIT-0 license. The original notebook is not redistributed. Its archive SHA-256 is

```
381e80382e8736aa00e18d8fd48c6f391b9ba0b15e80ff7621e7589147174930
```

The source notebook's SHA-256 is recorded separately in the source manifest. The article builds with a standard LaTeX distribution using two `pdflatex` passes. The validation output identifies the interpreter and SymPy versions actually used. Examples use zero-based Python root indices and explicitly distinguish them from the Wolfram indices in the motivating question.

References

- [1] V. Reshetnikov, *Why is ToRadicals[] not able to handle all cases? Is there a workaround?*, Mathematica Stack Exchange, question 34011 (2013). <https://mathematica.stackexchange.com/questions/34011/>. The accompanying `FindExtension.nb` was supplied with the task.
- [2] Wolfram Research, *ToRadicals*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/ToRadicals.html>. Accessed 8 September 2026.
- [3] J. S. Milne, *Fields and Galois Theory*, version 5.10, September 2022, especially the discussion of solvability by radicals in Chapter 5. <https://www.jmilne.org/math/CourseNotes/FT.pdf>.
- [4] C. Fieker and N. Sutherland, *Constructions using Galois Theory*, arXiv:2010.01281v3, 27 October 2022, especially Section 5. <https://arxiv.org/abs/2010.01281>.
- [5] SymPy Development Team, *Number Fields*, SymPy 1.14.0 documentation. <https://docs.sympy.org/latest/modules/polys/numberfields.html>. Accessed 8 September 2026.
- [6] SymPy Development Team, *Polynomials Manipulation Module Reference*, SymPy 1.14.0 documentation; `Poly.same_root` and exact root operations. <https://docs.sympy.org/latest/modules/polys/reference.html>. Accessed 8 September 2026.
- [7] Magma Computational Algebra System, *Galois Groups and Solving by Radicals*, handbook, `SolveByRadicals` and `CyclicToRadical`. <https://magma.maths.usyd.edu.au/magma/handbook/text/428>. Accessed 8 September 2026.